

P0333r0 : Improving Parallel Algorithm Exception Handling

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0333r0
Date: 2016-05-15
Reply-to: balelbach@lbl.gov
Author: Bryce Adelstein Lelbach
Contact: balelbach@lbl.gov
Audience: Library Evolution Working Group (LEWG)
Audience: Study Group 1 - Concurrency (SG1)
Audience: Library Working Group (LWG)
URL: <https://git.io/vrXCS>

1 Overview

The Parallelism Technical Specification (ISO/IEC TS 19570:2015) introduced a parallel algorithms library based on the existing standard algorithms library. At the February 2016 meeting in Jacksonville, the Parallelism TS was voted into the C++17 working draft.

The author of this paper has identified what he believes to be defects in the specification of the parallel algorithm exception handling (25.2.4, [algorithms.parallel.exceptions]).

2 Uncaught Element Access Function Exceptions Shouldn't Lead to terminate()

The exception handling behavior of parallel algorithms invoked with `par_vec` (the `parallel_vector_execution_policy`) is inconsistent with the exception handling behavior of the other two execution policies specified in the IS (`sequential` AKA `sequential_execution_policy` and `par` AKA `parallel_execution_policy`).

25.2.4 [algorithms.parallel.exception] states that if element access function exits via an uncaught exception in an parallel algorithm invoked under the `par_vec`, `terminate()` will be called. This is inconsistent with the other two policies, which would exit by throwing either the uncaught exception, or an `exception_list` containing (at least) the uncaught exception.

Additionally, the parallel algorithms may throw `bad_alloc` if they are unable to allocate the data structures needed to manage parallel execution - including `par_vec`. This adds another caveat to the error-reporting behavior of parallel algorithms - it is not as straightforward as " *sequential and par throw, par_vec doesn't throw*".

The exception handling behavior for `par_vec` makes it impossible to handle exceptions that occur in element access functions in a non-fatal fashion. The author of this paper feels that this behavior may hamper the usability of the parallel algorithms library.

The current behavior leads to a number of non-intuitive caveats when using parallel algorithms, complicates writing generic functions which use parallel algorithm and are templated on an `ExecutionPolicy` and adds complexity to the error-reporting guarantees of such generic functions,

Consider the following function:

```
template <class ExecutionPolicy, class It, class Compare, class Predicate>
It unique_sort(ExecutionPolicy&& policy, It first, It last, Compare comp, Predicate pred) noexcept
{
    sort(policy, first, last, comp);
    return unique(policy, first, last, pred);
}
```

What can we tell users of `unique_sort` about its error-reporting behavior? The answer is not simple. If `Compare` or `Predicate` throw an exception, `unique_sort` might throw an `exception_list` (or the underlying exception) or it might simply call `terminate()`. And, it could always potentially throw `bad_alloc`.

Now, suppose we have a second function, `unique_sorted()`, which constructs and returns a unique, sorted `vector<>` from a range of inputs.

```

template <class ExecutionPolicy, class It, class Compare, class Predicate>
vector<typename iterator_traits<It>::value_type>
unique_sorted(ExecutionPolicy&& policy, It first, It last, Compare comp, Predicate pred) noexcept
{
    using vector = vector<typename iterator_traits<It>::value_type>;

    try {
        sort(policy, first, last, comp);
        return vector(unique(policy, first, last, pred), last);
    } catch (...) {
        return vector();
    }
}

```

Our intention in the try-catch block should be clear - if an exception occurs, we catch it and return an empty vector<> (vector<>'s default constructor is noexcept). However, if unique_sorted() is invoked with par_vec, any exception thrown by Compare or Predicate will lead to terminate(), instead of the intended catch(...) block.

The current behavior of par_vec may also lead to non-intuitive resource leaks. Consider the following code, which utilizes a shared-memory mutex via Boost.Interprocess:

```

namespace ipc = boost::interprocess;

ipc::named_mutex mutex(ipc::open_or_create, "my_mutex");

ipc::scoped_lock<ipc::named_mutex> lock(mutex);

vector<double> d = // ...

try {
    for_each(par_vec, d.begin(), d.end(), my_function);
} catch (my_exception& me) {
    // ...
}

```

If my_function throws my_exception in the above code, terminate() will be called and the program will exit without invoking ipc::scoped_lock<> destructor, which will leave the shared-memory mutex in a locked state and cause a deadlock the next time this program is run.

Requiring par_vec to use the same exception-handling behavior as sequential and par may have one down-side. It could potentially limit the type of hardware resources that could be used to parallelize the execution of algorithms invoked with par_vec (some GPU platforms may have this limitation). In particular, some theoretical platform may not have a mechanism for performing stack unwinding (or, the mechanism may be quite slow). But, how would such a platform handle an exception which is thrown and caught *within* an element access function? If the platform can handle such exceptions in a conforming manner, then it should also be possible for that platform to support the exception-handling behavior of sequential and par. If the platform cannot handle such exceptions, then the current wording of par_vec is not a sufficient solution - it would be necessary to require that all element access functions used in par_vec invocations be noexcept.

The author believes the exception handling behavior of par_vec should be changed to be identical to the exception handling behavior of sequential and par.

3 Parallel Algorithms Should Always Throw exception_list or bad_alloc

If an element access function exits via an uncaught exception, a parallel algorithm invoked with sequential or par exit with either an exception_list or, optionally, the uncaught exception if only one uncaught exception occurred. Allowing different implementations to throw different exception types may cause portability problems and force users to duplicate catch blocks.

Implementations of parallel algorithms are not required to *"forge ahead"* in the face of exceptions. This relaxation gives implementations a great range of freedom in deciding whether to throw an exception_list or to simply throw the first uncaught exception from an element access function which is encountered.

```

try {
    for_each(par, d.begin(), d.end(), my_function);
} catch (exception_list& el) {
    // ...
} catch (my_exception& me) {
    // ...
}

```

For the above code snippet, different `catch()` blocks could be taken by different implementations. If `my_function` always threw an exception for each element, an implementation that does not "*forgo ahead*" would still be free to throw the underlying exception instead of an `exception_list`.

The author suggest that parallel algorithms should *always* report uncaught exceptions from element access functions via `exception_list`.

4 Proposed Wording Changes

Apply the following changes to 25.2.4 [algorithms.parallel.exceptions] paragraph 2:

During the execution of a parallel algorithm, if the invocation of an element access function exits via an uncaught exception, the behavior of the program is determined by the type of execution policy used to invoke the algorithm:

- If the execution policy object is of type `parallel_vector_execution_policy`, `terminate()` is called.
- If the execution policy object is of type `sequential_execution_policy`, or `parallel_execution_policy` or `parallel_vector_execution_policy`, the execution of the algorithm exits via an exception. The exception will be an `exception_list` containing all uncaught exceptions thrown during the invocations of element access functions, or optionally the uncaught exception if there was only one. [Note: For example, when `for_each` is executed sequentially, if an invocation of the user-provided function object throws an exception, `for_each` can exit via the uncaught exception, or throw an `exception_list` containing the original exception exception. - end note] [Note: These gurantees imply that, unless the algorithm has failed to allocate memory and exits via `bad_alloc`, all exceptions thrown during the execution of the algorithm are communicated to the caller. It is unspecified whether an algorithm implementation will "forgo ahead" after encountering and capturing a user exception. - end note] [Note: The algorithm may exit via the `bad_alloc` exception even if one or more user-provided function objects have exited via an exception. For example, this can happen when an algorithm fails to allocate memory while creating or adding elements to the `exception_list` object. - end note]
- If the execution policy object is of any other type, the behavior is implementation-defined.